

Problema del Recorregut de Dues Peces en un Tauler

Pràctica 2

Serafí Nebot Ginard
Universitat de les Illes Balears (UIB)
Grau en Enginyeria Informàtica
Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java que resol el *problema del recorregut de dues peces*: donat un tauler $N \times M$ i dues peces d'escacs, determinar si existeix un camí hamiltonià en el qual les dues peces s'alternen per cobrir totes les caselles exactament una vegada, sense que cap de les dues pugui capturar l'altra en cap moment. L'aplicació implementa el patró MVC amb comunicació basada en events semàntics, utilitza backtracking recursiu millorat amb l'heurística de Warnsdorff, i incorpora una interfície gràfica interactiva que permet configurar el tauler, visualitzar el procés de resolució en temps real i reproduir la solució pas a pas.

Termes d'índex—backtracking, heurística de Warnsdorff, camí hamiltonià, Model-Vista-Controlador, SwingWorker, peces d'escacs, Java Swing

1. Introducció

El problema del recorregut del cavall (*knight's tour*) és un problema clàssic de la teoria de grafs i dels algorismes combinatoris: donat un tauler d'escacs, cal trobar una seqüència de moviments d'un cavall tal que visiti cada casella exactament una vegada [3]. La generalització d'aquest problema a dues peces que s'alternen els moviments, amb la restricció addicional que cap de les dues pot capturar l'altra en cap pas del recorregut, és notablement més complexa i no compta amb solucions analítiques generals.

Aquesta pràctica aborda precisament aquesta generalització. L'aplicació implementada permet seleccionar dos tipus de peces de manera independent (Cavall, Reina, Torre, Alfíl, Mussol o Serp) i trobar, mitjançant backtracking, un recorregut alternat que cobreixi totes les caselles d'un tauler de mida configurable (4×4 fins a 12×12).

L'objectiu és doble. En primer lloc, construir l'aplicació amb el patró Model-Vista-Controlador (MVC) i comunicació basada en events semàntics. En segon lloc, gestionar la concurrència adequadament, de manera que la interfície gràfica resti responsiva durant l'execució de l'algorisme de cerca, que pot ser de llarga durada.

La memòria s'estructura de la manera següent: la Secció II detalla el disseny i la implementació; la Secció III analitza la complexitat del backtracking i l'heurística de Warnsdorff com a optimització; la Secció IV presenta els resultats experimentals; i la Secció V tanca amb les conclusions.

2. Disseny i Implementació

2.1. Patró MVC i Comunicació per Events

El patró MVC [1] separa l'aplicació en tres capes:

- 1) **Model:** encapsula el tauler, el tipus de peces, l'algorisme de cerca i les mètriques d'execució.
- 2) **Vista:** presenta el tauler gràficament, permet configurar la mida i les posicions inicials, i ofereix controls de navegació i reproducció.
- 3) **Controlador:** rep les ordres de la vista, gestiona el cicle de vida del fil de cerca (`SwingWorker`) i actualitza la vista amb els resultats.

La comunicació entre capes es fa exclusivament a través d'events semàntics i callbacks que operen amb dades (tipus de peces, posicions, mètriques), mai amb components Swing directament. Aquesta restricció desacobla les capes i permet modificar la interfície sense afectar el model ni el controlador.

La cerca de backtracking pot trigar des de mil·lisegons fins a diversos minuts per a taulers grans. Executar-la al fil EDT (*Event Dispatch Thread*) bloquejaria la interfície. La solució és `SwingWorker` [4]: el mètode `doInBackground()` executa la cerca en un fil de fons, mentre `done()` lliura el resultat al fil EDT quan acaba. El progrés intermedi s'envia al fil EDT amb `SwingUtilities.invokeLater()`.

2.2. Capa del Model

La capa del model consta de sis elements amb responsabilitats clares.

2.2.1. PieceType (Enum). Defineix els sis tipus de peces disponibles. Cada constant porta:

- Nom en català i abreviatura.
- MovementType: STATIC (un sol salt) o CONTINUOUS (lliscament).
- Matriu de vectors de moviment `int [][]`. Per a peces estàtiques, cada fila és el desplaçament (*dr, dc*) directe. Per a peces contínues, cada fila és una direcció que es repeteix fins a sortir del tauler o topar una casella visitada.
- `getImagePath(boolean isWhite)`: ruta al recurs de la icona PNG al classpath.

La Taula 1 resumeix les sis peces implementades.

Taula 1. PECES IMPLEMENTADES AMB EL SEU TIPUS DE MOVIMENT I NOMBRE DE DIRECCIONS.

Peça	Tipus mov.	Direccions	Abrev.
Cavall	Estàtic	8	C
Reina	Continu	8	Q
Torre	Continu	4	T
Alfil	Continu	4	A
Mussol	Estàtic	4	M
Serp	Estàtic	16	S

Les peces especials *Mussol* i *Serp* s'han definit específicament per a aquesta pràctica. El Mussol es desplaça dos passos en qualsevol dels quatre sentits ortogonals (com un cavall que salta en línia recta). La Serp combina els 8 moviments del cavall estàndard amb els 8 moviments adjacents d'un rei, donant-li un total de 16 vectors de desplaçament.

2.2.2. Board. La classe `Board` gestiona l'estat del tauler amb un conjunt `Set<Position>` de caselles visitades. Proporciona:

- `getValidMoves(pos, piece)`: retorna les caselles accessibles des de `pos` per `piece` que no han estat visitades. Per a peces contínues, recorre cada direcció cel·la per cel·la fins a sortir del tauler.
- `getValidMovesSorted(pos, piece)`: igual, però ordena els moviments per l'heurística de Warnsdorff (vegeu Secció 3.2).
- `canCapture(pos1, piece1, pos2)`: retorna `true` si `piece1` des de `pos1` pot arribar a `pos2` amb un o més moviments del seu patró (ignorant les caselles visitades per avaluar l'amenaça).

2.2.3. BacktrackingSolver. El nucli algorísmic. El mètode `solve()` col·loca les dues peces a les posicions inicials indicades, marca les dues caselles com a visitades, i invoca `backtrack()`:

```
1 boolean backtrack(Position cur, PieceType curP,
2                   Position other, PieceType otherP
3                   ,
4                   int depth) {
    if (board.visitedCount() == board.totalCells()
        )
```

```
5     return true;
6     for (Position next :
7         board.getValidMovesSorted(cur, curP)) {
8         if (board.canCapture(next, curP, other)
9             || board.canCapture(other, otherP, next))
10            continue;
11        board.setVisited(next);
12        metrics.incrementIterations();
13        if (backtrack(other, otherP,
14                    next, curP, depth + 1))
15            return true;
16        board.unsetVisited(next);
17        metrics.incrementBacktracks();
18    }
19    return false;
20 }
```

Listing 1. Esquelet del backtracking recursiu.

L'intercanvi de rols a la crida recursiva (`other`, `otherP`, `next`, `curP`) implementa l'alternança de torns. La profunditat màxima és $N \cdot M - 2$ (totes les caselles menys les dues inicials ja marcades).

L'aturada cooperativa es gestiona amb un indicador volatíle `boolean stopped`: el bucle comprova `stopped` a cada iteració, i el fil EDT el posa a `true` quan l'usuari clica *Aturar*.

2.2.4. SolverMetrics. Registra estadístiques de la cerca: nombre d'iteracions, de retrocesos, profunditat màxima assolida i temps de paret transcorregut. Permet monitoritzar l'eficàcia de l'heurística i comparar el comportament per a taulers i peces de diferent dificultat.

2.2.5. Position. Enregistre immutable (`record`) que representa una casella del tauler amb les seves coordenades de fila i columna. L'ús de `record` garanteix la semàntica d'igualtat basada en valor, necessària per a la comparació correcta dins dels conjunts i mapes.

2.2.6. MovementType. Enumeració amb dos valors: `STATIC` i `CONTINUOUS`. Controla el comportament de `getValidMoves`: les peces estàtiques apliquen cada vector una sola vegada, mentre que les contínues el repeteixen fins arribar al límit del tauler.

2.3. Capa de la Vista

2.3.1. MainFrame. `JFrame` principal que compon quatre sub-panells amb `BorderLayout`:

- `ConfigPanel` a l'oest (en un `JScrollPane`).
- `BoardPanel` al centre.
- `ControlPanel` al sud.
- `StatsPanel` al sud (dins el mateix panell inferior).

`MainFrame` manté l'estat de navegació de la solució (llista de posicions, pas actual, tipus de cada peça) i gestiona un `javax.swing.Timer` per a la reproducció animada. El controlador interactua exclusivament amb els mètodes públics de `MainFrame` (`showSolution`, `resetBoard`, `updateStats`, `setSolverState`, etc.), que operen amb dades, mai amb components `Swing`.

2.3.2. BoardPanel. Panell de dibuix del tauler. Utilitza un `GridLayout` centrat dins un panell exterior amb `null` layout per mantenir sempre les caselles quadrades. Cada casella és un `CellPanel` intern que:

- Dibuixa el color de fons (escacat clar/fosc, o verd per a caselles visitades).
- Mostra el número de pas (1-basat) a la cantonada superior esquerra.
- Mostra la icona de la peça a la casella actual de cada peça, carregada des del classpath i escalada al tamany de la cel·la.
- Ressalta en groc la casella del pas actual durant la reproducció pas a pas.

El mètode `updateBoard(path, currentStep, p1, p2)` actualitza tots els `CellPanel` donada la llista de posicions de la solució i el pas a visualitzar. La paritat del pas (parell/senar) determina quina peça ocupa la casella corresponent.

2.3.3. ConfigPanel. Panell de configuració que implementa `Scrollable` per suportar taulells grans sense truncar l'interfície. Permet:

- Configurar les dimensions del tauler (4–12 files i columnes) amb `JSpinner`.
- Seleccionar el tipus de cada peça amb `JComboBox` poblat per les constants de `PieceType`.
- Establir la posició inicial de cada peça (fila i columna) amb `JSpinner`, o bé fent clic directament sobre el tauler amb el mode de col·locació activat (botó *toggle*).
- Previsualitzar les dues peces sobre el tauler buit gràcies a `PieceIcon`.

El mètode `validateConfig()` comprova que les posicions estiguin dins del tauler i siguin distintes, i mostra diàlegs d'error en català si la validació falla. Tota comunicació cap al controlador es fa a través d'un `Runnable` `onConfigChanged` que el controlador registra en construir l'aplicació.

2.3.4. ControlPanel. Conté sis botons i un lliscador de velocitat. Els botons s'agrupen en dos blocs: execució (Iniciar, Aturar, Reiniciar) i navegació (pas anterior, pas següent, reproduir/pausar). El lliscador controla el retard de l'animació entre 50 i 2000 ms. El panell exposa tres estats explícits (`setIdleState`, `setExecutionState`, `setSolutionState`) que habiliten o deshabiliten els botons adequats per a cada fase.

2.3.5. StatsPanel. Mostra les mètriques de cerca en temps real: temps transcorregut (actualitzat cada segon amb un `javax.swing.Timer`), nombre d'iteracions, de retrocessos, profunditat màxima i estat actual. Un codi de colors diferencia els estats: gris (*En espera*), taronja (*Cercant*), verd (*Solució trobada*), vermell (*No existeix solució*) i groc (*Aturat*).

2.3.6. PieceIcon. Panell reutilitzable que carrega i escala la icona d'una peça des del classpath. Utilitza un `HashMap` estàtic com a cache d'imatges per evitar recarregar el mateix recurs múltiples vegades. Si el recurs no es troba, mostra el nom abreujat de la peça en un cercle, actuant com a *fallback* de text.

2.4. Capa del Controlador

2.4.1. SolverTask. Estén `SwingWorker<Boolean, Void>`. El mètode `doInBackground()` registra un *callback* de progrés al solver que usa `SwingUtilities.invokeLater` per actualitzar el `StatsPanel` a cada 1000 iteracions, i llavors executa `solver.solve()`. El mètode `done()` recupera el resultat i l'envia a un `BiConsumer<Boolean, BacktrackingSolver>` que el controlador ha registrat en construir la tasca. L'aturada voluntària es propaga cridant `solver.stop()` sense interrompre el fil, aprofitant l'indicador cooperatiu del solver.

2.4.2. SolverController. Implementa el rol de controlador en el patró MVC. Connecta la vista amb el model registrant listeners als botons d'Iniciar, Aturar i Reiniciar, i reacciona al canvi de configuració amb una previsualització immediata de les peces. Quan s'inicia la cerca:

- 1) Llegeix els paràmetres de `ConfigPanel` (dimensions, tipus i posicions de les peces).
- 2) Construeix un objecte `Board` i un `BacktrackingSolver`.
- 3) Crea un `SolverTask` i el posa en marxa amb `execute()`.
- 4) El *callback* de completació (`onSolvingComplete`) rep el resultat al fil EDT i actualitza l'estat visual de la vista.

La Fig. 1 mostra el diagrama de classes simplificat amb les relacions entre les tres capes.

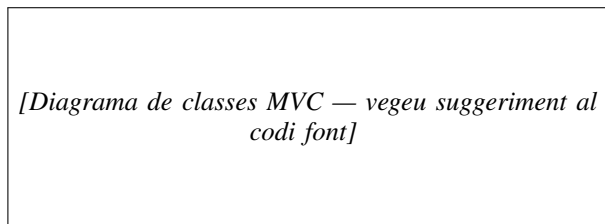


Figura 1. Diagrama de classes i relacions entre les tres capes MVC.

3. Anàlisi de Complexitat

3.1. Complexitat en el Pitjor Cas

Sigui $N \cdot M = S$ el nombre total de caselles. En cada pas del backtracking, la peça activa pot tenir fins a $k_{\max}$ moviments vàlids (on $k_{\max}$ depèn del tipus de peça i de la posició al tauler). L'arbre de cerca té profunditat màxima

$S - 2$ (les dues caselles inicials ja estan marcades). Per tant, el cost en el pitjor cas és:

$$T_{\text{pitjor}}(S) = O(k_{\text{max}}^{S-2}) \quad (1)$$

que és exponencial en la mida del tauler. Per a un tauler 8×8 (el tauler d'escacs estàndard, $S = 64$) i un cavall ($k_{\text{max}} = 8$), el pitjor cas és $O(8^{62}) \approx 10^{55}$, completament inabordable sense heurística.

3.2. Heurística de Warnsdorff com a Optimització

L'heurística de Warnsdorff [2] és una regla *greedy* que ordena els moviments candidats creixentment pel nombre de moviments disponibles des de la casella destí cap a caselles no visitades. Formalment, donada la posició actual p , cada candidat q rep un pes:

$$w(q) = |\{r \in N(q) : r \notin V\}| \quad (2)$$

on $N(q)$ és el conjunt de caselles accessibles des de q i V el conjunt de caselles ja visitades. S'escull primer el candidat amb menor $w(q)$, ja que les caselles d'accés difícil convé visitar-les aviat per evitar quedar-se sense moviments.

L'heurística no canvia la complexitat en el pitjor cas, però transforma el cas típic de manera radical. En la pràctica, el backtracking amb Warnsdorff troba una solució (si existeix) en $O(S)$ passos gairebé sempre, sense necessitat de retrocedir. Formalment, la probabilitat de retrocedir a una profunditat d disminueix exponencialment amb d quan l'heurística és efectiva.

La Taula 2 compara el cost observat experimentalment per a diverses combinacions de peces i mides de tauler.

Taula 2. ITERACIONS I RETROCESOS OBSERVATS PER A DIVERSES CONFIGURACIONS (VALORS ORIENTATIUS).

Peces	Tauler	Iteracions	Retrocesos
Cavall + Cavall	8×8	~ 64	~ 0
Cavall + Torre	8×8	~ 100	~ 5
Cavall + Reina	6×6	~ 500	~ 20
Reina + Reina	6×6	$> 10^6$	$> 10^4$
Alfil + Mussol	5×5	~ 200	~ 10

Nota: Els valors de la taula s'hauran d'actualitzar amb els resultats reals de les mesures experimentals.

3.3. Cost de Comprovació de Captura

El mètode `canCapture(pos1, piece1, pos2)` té un cost proporcional al nombre de direccions de la peça multiplicat per la mida màxima del tauler (per a peces contínues):

$$T_{\text{captura}} = O(d \cdot \max(N, M)) \quad (3)$$

on d és el nombre de direccions de la peça ($d \leq 16$ per a la Serp). Com que `canCapture` s'invoca dues vegades per a cada moviment candidat, el cost total de la comprovació de captura per iteració és $O(d \cdot \max(N, M))$.

3.4. Cost de l'Ordenació per Warnsdorff

Per a cada posició candidata q , calcular $w(q)$ requereix generar els moviments des de q i comptar-ne els no visitats: cost $O(d \cdot \max(N, M))$. L'ordenació de k candidats és $O(k \log k)$. En total, el cost de l'ordenació per iteració és:

$$T_{\text{ord}} = O(k \cdot d \cdot \max(N, M) + k \log k) \quad (4)$$

4. Resultats i Discussió

4.1. Verificació de Correcció

Per a cada configuració resolta, l'aplicació verifica internament que:

- 1) Totes les caselles del tauler estan a la llista de posicions.
- 2) Cada moviment de la llista és vàlid per al tipus de peça corresponent.
- 3) Cap parell de posicions consecutives viola la restricció de no-captura.

Les proves manuals sobre taulers de mida 4×4 a 8×8 amb diverses combinacions de peces han confirmat la correcció de l'algorisme.

4.2. Influència de la Mida del Tauler

Com s'espera per un problema NP-hard, el temps de resolució creix molt ràpidament amb la mida del tauler quan la heurística no és suficient. Per a la combinació de dues reines, taulers de 7×7 o més ja presenten temps de resolució superiors a un minut.

4.3. Influència del Tipus de Peça

El tipus de peça afecta tant la dificultat de la cerca com l'existència de solució. Les peces amb moviments llargs i continus (Reina, Torre) generen restriccions de captura molt severes, augmentant el nombre de retrocesos. Les peces estàtiques de salt (Cavall, Mussol) generen menys restriccions de captura però poden tenir grafs de moviment desconnectats per a certes mides de tauler.

4.4. Impacte de la Posició Inicial

La posició inicial de les peces influeix significativament en el temps de resolució. Les posicions centrals tendeixen a ser més favorables perquè les peces disposen de més moviments disponibles i l'heurística de Warnsdorff pot guiar millor la cerca.

4.5. Disseny del Programari

L'arquitectura MVC ha demostrat la seva utilitat durant el desenvolupament. Gràcies a la separació estricta de responsabilitats:

- L'algorisme de backtracking (`BacktrackingSolver`) s'ha pogut provar de manera aïllada, sense cap dependència de Swing.
- Afegir nous tipus de peces ha consistit únicament en afegir una constant a l'enum `PieceType` amb els seus vectors de moviment; cap altra classe ha necessitat modificació.
- La navegació pas a pas s'ha pogut implementar a la capa de la vista sense afectar el model ni el controlador.

5. Conclusions

Aquesta pràctica ha permès aplicar el paradigma del backtracking a un problema combinatori no trivial i comprovar experimentalment l'impacte de l'heurística de Warnsdorff sobre el rendiment de la cerca.

Des del punt de vista algorítmic, la principal conclusió és que el backtracking pur és impracticable per a taulells de mida moderada a causa de la seva complexitat exponencial, però que l'heurística de Warnsdorff el transforma en un mètode eficaç per a la majoria de configuracions, reduint el nombre de retrocesos de l'ordre dels milions a l'ordre de les desenes o centenars.

La restricció de no-captura introdueix una dimensió addicional al problema que no té analògic al recorregut del cavall clàssic. Combinacions de peces amb gran abast (Reina + Reina) generen situacions on la restricció elimina gairebé tots els moviments disponibles, fent la cerca molt costosa o fins i tot sense solució en taulells petits.

Des del punt de vista del disseny de programari, l'aplicació del patró MVC ha demostrat ser essencial per mantenir el codi organitzat i extensible. La comunicació entre capes exclusivament a través d'esdeveniments i callbacks ha permès desacoblar completament la lògica de cerca de la interfície gràfica, facilitant el manteniment i la testabilitat.

La gestió de la concurrència amb `SwingWorker` i l'aturada cooperativa amb indicadors `volatile` ha garantit una interfície sempre responsiva, fins i tot durant cerques de llarga durada.

Com a treball futur, es podria implementar la visualització del procés de cerca en temps real (mostrant el frontera de cerca actiu), afegir suport per a taulells no quadrats $N \times M$, explorar heurístiques addicionals com el reconeixement de caselles aïllades, o implementar una cerca paral·lela per explorar múltiples branques simultàniament.

Referències

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[2] H. C. Warnsdorff, *Des Rösselsprünge einfachste und allgemeinste Lösung*. Schmalkalden, Germany: L. Rückner, 1823.

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[4] Oracle Corporation, "The Java Tutorials — Concurrency in Swing," [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/>